

DevOps - Giới thiệu

DevOps = Development (Dev) + Operations (Ops)

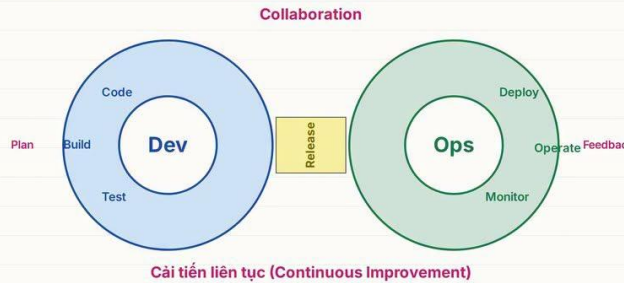
DevOps là một văn hóa và tập hợp các phương pháp thực hành giúp gắn kết đội Development và đội Operations lại với nhau, nhằm phát hành phần mềm nhanh hơn, đáng tin cậy hơn và với chất lượng cao.

Vì sao cần DevOps?

- ✓ Phát hành nhanh hơn (Faster Delivery)
- ✓ Chất lượng cao hơn
- ✓ Tỷ lệ lỗi thấp hơn
- ✓ Cải thiện sự hợp tác (Collaboration)
- ✓ Sự hài lòng của khách hàng ☆

1. DevOps là gì?

- DevOps không phải là một công cụ (tool), mà là một Mindset và Culture (tư duy và văn hóa).
- Nó nhằm rút ngắn SDLC và mang lại continuous delivery với chất lượng phần mềm cao.
- Nó phá vỡ các rào cản (silos) giữa đội Development và đội Operations.
- Nó sử dụng automation, monitoring và collaboration để cải thiện quy trình phát hành phần mềm.



2. Mục tiêu của DevOps

1. Continuous Delivery / Deployment
2. Tự động hóa các quy trình
3. Continuous Integration
4. Continuous Monitoring
5. Phản hồi và cải tiến nhanh chóng

3. Nguyên tắc của DevOps

- ☆ Customer-Centric Approach
- ☆ Collaboration & Communication
- ☆ Automation Everything
- ☆ Continuous Integration & Delivery
- ☆ Monitor Everything
- ☆ Infrastructure as Code
- ☆ Continuous Feedback

4. Traditional vs DevOps

Cách tiếp cận truyền thống	Khía cạnh	Cách tiếp cận DevOps
Các đội tách biệt (Dev & Ops riêng)	Cấu trúc đội	Đội đa chức năng (Dev + Ops cùng nhau)
Quy trình thủ công	Quy trình	Quy trình tự động hóa
Chu kỳ phát hành dài (hàng tháng)	Tần suất phát hành	Chu kỳ phát hành ngắn (ngày/giờ)
Tập trung vào mục tiêu cá nhân	Trọng tâm	Tập trung vào giá trị khách hàng
Nhiều lỗi và sự cố hơn	Chất lượng	Ít lỗi hơn, chất lượng cao
Bị động (sửa sau khi lỗi xảy ra)	Giám sát	Chủ động (giám sát và cải tiến)

5. Các thực hành chính trong DevOps

- **Continuous Integration (CI)**: Developer tích hợp code thường xuyên vào một shared repository.
- **Continuous Delivery (CD)**: Code luôn ở trạng thái sẵn sàng deploy và có thể release bất cứ lúc nào.
- **Infrastructure as Code (IaC)**: Quản lý hạ tầng bằng code (ví dụ: Terraform, Ansible).
- **Automation**: Tự động hóa các tác vụ lặp lại như build, test, deploy, monitoring.
- **Monitoring & Logging**: Giám sát application, hạ tầng và thu thập log để phân tích.

6. Lợi ích của DevOps

- Rút ngắn thời gian đưa sản phẩm ra thị trường
- Cải thiện sự hợp tác
- Tăng năng suất làm việc
- Chất lượng phần mềm tốt hơn
- Khả năng mở rộng và ổn định
- Giảm chi phí

DevOps là một hành trình, không phải là đích đến. ☺

7. Công cụ DevOps (Ví dụ)

Version Control	CI/CD Tools	Build Tools	Containerization	Orchestration	IaC Tools	Monitoring Tools
Git, GitHub, GitLab, Bitbucket	Jenkins, GitLab CI, GitHub Actions,...	Maven, Gradle, Ant	Docker, Podman	Kubernetes, Docker Swarm	Terraform, Ansible, Puppet,...	Prometheus, Grafana, ELK



DevOps - Kiến thức nền tảng Linux

Linux là xương sống của DevOps

Linux là gì?

Linux là một hệ điều hành mã nguồn mở (open source), giống Unix (Unix-like). Nó được sử dụng rộng rãi trong server, cloud, container và môi trường DevOps vì tính bảo mật, ổn định và linh hoạt. ☆

1. Các bản phân phối Linux

Một số bản phân phối Linux phổ biến dùng trong DevOps:

- Ubuntu
- CentOS / AlmaLinux / Rocky Linux
- Debian
- Red Hat Enterprise Linux (RHEL)
- Amazon Linux

2. Cấu trúc hệ thống tệp (File System)

```
/ (root)
├ /bin      = Các lệnh nhị phân thiết yếu
├ /sbin     = Lệnh hệ thống (dành cho admin)
├ /etc      = Các tệp cấu hình
├ /home     = Thư mục home của người dùng
├ /var      = Tệp biến đổi (log, spool, cache)
├ /tmp      = Tệp tạm thời
├ /usr      = Chương trình và tệp người dùng
├ /opt      = Ứng dụng bổ sung tùy chọn
├ /boot     = Tệp boot loader
├ /dev      = Tệp thiết bị (device)
├ /proc     = Thông tin process & kernel (ảo)
├ /root     = Thư mục home của user root
├ /lib      = Thư viện dùng chung thiết yếu
├ /mnt      = Điểm mount tạm thời
├ /media    = Điểm mount cho thiết bị di động
└ /srv      = Dữ liệu dịch vụ (service)
```

3. Các lệnh cơ bản

Lệnh	Ví dụ	Mô tả
pwd	pwd	In ra thư mục làm việc hiện tại
ls	ls -l	Liệt kê tệp và thư mục
cd	cd /home/user	Chuyển thư mục
mkdir	mkdir dir1	Tạo một thư mục
rmdir	rmdir dir1	Xóa thư mục rỗng
rm	rm file.txt	Xóa một tệp
cp	cp src.txt dest.txt	Sao chép tệp
mv	mv old.txt new.txt	Di chuyển hoặc đổi tên tệp
cat	cat file.txt	Hiển thị nội dung tệp
less	less file.txt	Xem nội dung tệp theo từng trang
head	head -n 10 file.txt	10 dòng đầu của tệp
tail	tail -n 10 file.txt	10 dòng cuối của tệp
touch	touch file.txt	Tạo một tệp rỗng
which	which python3	Vị trí của một lệnh
man	man ls	Hướng dẫn sử dụng của một lệnh
clear	clear	Xóa màn hình terminal

8. Quản lý người dùng

- whoami — User đang đăng nhập hiện tại
- id — ID của user và group
- adduser username — Thêm user mới (Ubuntu/Debian)
- useradd username — Thêm user mới (RHEL/CentOS)
- passwd username — Đặt hoặc đổi mật khẩu
- usermod -aG group user — Thêm user vào group
- userdel username — Xóa một user

4. Thao tác với tệp

Lệnh	Mô tả
ls -l	Liệt kê chi tiết (quyền, kích thước, ngày)
ls -a	Hiển thị tệp ẩn
ls -lh	Kích thước dễ đọc (human readable)
cp -r dir1 dir2	Sao chép thư mục đệ quy
mv file1 file2	Đổi tên file1 thành file2
rm -r dir	Xóa thư mục đệ quy
rm -f file	Buộc xóa tệp
find /path -name "*.txt"	Tìm tệp theo tên
grep "text" file.txt	Tìm chuỗi text trong tệp
wc -l file.txt	Đếm số dòng trong tệp

5. Phân quyền tệp (File Permissions)

Ký hiệu quyền	Biểu diễn số
- rwx rwx rwx	rwx = 4+2+1 = 7
user group other	rw- = 4+2 = 6
r = read (4)	r-x = 4+1 = 5
w = write (2)	r-- = 4
x = execute (1)	-wx = 2+1 = 3
	--x = 1
	--- = 0

Ví dụ chmod

- chmod 755 file.sh — rwxr-xr-x
- chmod 644 file.txt — rw-r--r--
- chmod +x file.sh — thêm quyền execute
- chmod -x file.sh — bỏ quyền execute
- chown user:group file.txt — đổi owner và group

6. Quản lý tiến trình (Process)

- ps aux — Hiển thị tất cả process đang chạy
- top — Theo dõi process theo thời gian thực
- htop — Xem process dạng tương tác (nếu đã cài)
- kill <PID> — Kill process theo PID
- kill -9 <PID> — Buộc kill process

7. Lệnh thông tin hệ thống

- uname -a — Thông tin hệ thống và kernel
- hostname — Xem hoặc đặt hostname
- df -h — Dung lượng đĩa còn trống
- du -sh /path — Kích thước của thư mục
- free -h — Dung lượng bộ nhớ (memory)
- uptime — Thời gian hệ thống hoạt động và load

9. Lệnh mạng (Networking)

- ip a / ifconfig — Hiển thị địa chỉ IP và interface
- ping google.com — Kiểm tra kết nối mạng
- netstat -tuln — Hiển thị port đang mở và service đang lắng nghe
- ss -tuln — Cách khác để xem port đang mở
- curl google.com — Lấy phản hồi từ URL
- wget https://site.com — Tải tệp từ URL

GiT - Công cụ quản lý phiên bản (Version Controlling Tool)

1. Tìm hiểu về Version Control

- Version Control System (VCS) ghi lại các thay đổi của tệp theo thời gian để bạn có thể gọi lại phiên bản cụ thể sau này.
- Nó hỗ trợ collaboration, backup, theo dõi lịch sử và revert lại lỗi.
- Các loại: Centralized VCS (vd: SVN), Distributed VCS (vd: Git)

Theo dõi thay đổi
Cộng tác tốt hơn ☺

Cú pháp CLI cơ bản

git <command> <options> <arguments>

Ví dụ:

```
git status
git add file.txt
```

2. Git - Một công cụ CLI

Git là một công cụ Command Line Interface (CLI) dùng để tương tác với Git repository.

Vì sao dùng CLI?

- ✓ Nhanh và nhẹ
- ✓ Mạnh mẽ và linh hoạt
- ✓ Hoạt động trên mọi hệ điều hành lớn
- ✓ Nền tảng cho mọi Git GUI

Git là Distributed Version Control System phổ biến nhất. ☆

3. Vai trò cốt lõi của Git trong ngành

- ☆ Cộng tác trong team
- ☆ Branching cho feature & release
- ☆ Backup và khôi phục code
- ☆ Tích hợp với CI/CD pipeline
- ☆ Theo dõi thay đổi và lịch sử
- ☆ Hỗ trợ DevOps & Agile workflow

4. Cách thiết lập Git

4.1 Cài đặt Git

- Tải về từ <https://git-scm.com/downloads>
- Cài đặt cho hệ điều hành của bạn (Windows/macOS/Linux)
- Kiểm tra cài đặt: git --version

4.3 Lấy một Git Repository

Tạo Repository mới

```
mkdir my-project
cd my-project
git init
(Tạo ra thư mục .git)
```

Clone Repository có sẵn

```
git clone <repo-url>
cd <repo-name>
```

Repository (repo) là một thư mục project được Git theo dõi. ☺

Git - Các lệnh & Workflow

5. Làm việc với các lệnh Git khác nhau

<code>git init</code>	→	Khởi tạo một repository mới
<code>git clone <url></code>	→	Clone một repository có sẵn
<code>git status</code>	→	Kiểm tra trạng thái của các tệp
<code>git add <file></code>	→	Thêm tệp vào staging area
<code>git commit -m "msg"</code>	→	Commit thay đổi kèm message
<code>git log</code>	→	Hiển thị lịch sử commit
<code>git branch</code>	→	Liệt kê các branch
<code>git checkout <branch></code>	→	Chuyển sang branch khác
<code>git merge <branch></code>	→	Merge branch vào branch hiện tại
<code>git remote -v</code>	→	Liệt kê các remote
<code>git push</code>	→	Đẩy thay đổi lên remote
<code>git pull</code>	→	Kéo thay đổi từ remote về

6. Ghi lại các thay đổi vào Repository

① Kiểm tra trạng thái các tệp của bạn	<code>git status</code>
② Theo dõi tệp mới	<code>git add <file></code>
③ Staging các tệp đã sửa đổi	<code>git add .</code>
④ Bỏ qua tệp khỏi GIT Tạo tệp .gitignore Thêm pattern (vd: *.log, node_modules/) Git sẽ bỏ qua các tệp đó.	
⑤ Xem thay đổi Unstaged và Staged	<code>git diff (unstaged)</code> <code>git diff --staged</code>
⑥ Cách commit thay đổi của bạn	<code>git commit -m "message"</code>
⑦ Bỏ qua Staging Area và commit	<code>git commit -am "message"</code>
⑧ Xóa tệp khỏi GIT	<code>git rm <file></code>

7. Xem lịch sử Commit

`git log` → Toàn bộ lịch sử

7.1 Giới hạn kết quả Log

<code>git log --oneLine</code>	→ Mỗi commit một dòng
<code>git log -n 5</code>	→ 5 commit gần nhất
<code>git log --since="7 days ago"</code>	→ Commit trong 7 ngày qua
<code>git log --author="name"</code>	→ Commit theo một author

7.2 Dùng GUI để trực quan hóa lịch sử ☆

Công cụ: Git GUI, SourceTree, GitKraken, VS Code

Các công cụ này hiển thị branch, commit, diff theo dạng đồ họa.



Git - Undoing, Branching & Workflow

8. Hoàn tác thay đổi (Undoing Things)

8.1 Thay đổi Commit cuối cùng

- Nếu chưa push: `git commit --amend`
- Đổi message hoặc thêm tệp vào commit cuối.

8.2 Unstage một tệp đã Staged

`git restore --staged <file>` (hoặc) `git reset HEAD <file>`

8.3 Hoàn tác thay đổi trên tệp đã sửa

`git restore <file>`

(Loại bỏ thay đổi trong working directory)

9. Làm việc với Remote

9.1 Hiện thị Remote của bạn — `git remote -v`

9.2 Thêm Remote Repository — `git remote add origin <url>`

9.3 Fetch và Pull từ Remote:

- `git fetch origin` → Tải thay đổi về
- `git pull origin <branch>` → Fetch + Merge

9.4 Push lên Remote — `git push -u origin <branch>`

9.5 Kiểm tra một Remote — `git remote show origin`

9.6 Xóa và đổi tên Remote:

- Xóa: `git remote remove origin`
- Đổi tên: `git remote rename origin upstream`

10. Branching và Merging trong Git

10.1 Branch là gì

Một branch là con trỏ (pointer) nhẹ, có thể di chuyển, trỏ đến một commit. Nó cho phép bạn làm việc trên các feature khác nhau một cách độc lập.

10.2 Cơ bản về Branching và Merging

- `git branch <name>` → Tạo branch
- `git checkout <name>` → Chuyển branch
- `git checkout -b <name>` → Tạo & chuyển
- `git merge <branch>` → Merge vào branch hiện tại

10.3 Quản lý Branch trong Git

- `git branch` → Liệt kê branch
- `git branch -d <name>` → Xóa branch (an toàn)
- `git branch -D <name>` → Buộc xóa branch

10.4 Branching Workflow và cách dùng

- Git Flow: Feature, Develop, Release, Hotfix, Main
- GitHub Flow: Đơn giản, branch từ main, PR & merge
- Trunk Based Development: Branch tồn tại ngắn hạn

10.5 Remote Branch - Tạo và Xóa

- Tạo: `git push -u origin <branch>`
- Liệt kê Remote: `git branch -r`
- Xóa Remote: `git push origin --delete <branch>`

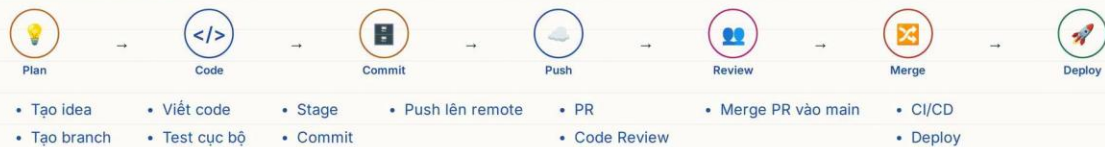
10.6 Rebase

`git checkout <branch>`
`git rebase <base-branch>`

(Áp lại commit lên trên base branch) → Giữ lịch sử sạch sẽ.

Rebase viết lại lịch sử, hãy cẩn thận khi dùng trên branch dùng chung! ☹

11. Git Workflow





Tự động hóa Build, Test và Deploy một cách nhất quán! ☺

JENKINS

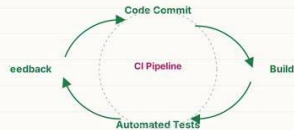
Jenkins là một automation server mã nguồn mở, dùng để build, test và deploy ứng dụng liên tục trong môi trường DevOps. ☆

1. Yếu tố cốt lõi của Continuous Integration

Continuous Integration (CI) là một thực hành DevOps, trong đó developer tích hợp code vào một shared repository thường xuyên. Mỗi lần tích hợp sẽ kích hoạt build và test tự động để phát hiện lỗi sớm.

Lợi ích chính:

- Phát hiện lỗi sớm
- Cải thiện chất lượng code
- Giảm thời gian và công sức tích hợp
- Tăng năng suất developer
- Phản hồi (feedback) nhanh hơn
- Cho phép release thường xuyên và đáng tin cậy



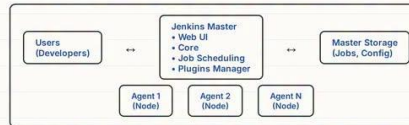
2. Một ví dụ tình huống sử dụng CI



3. Tìm hiểu chi tiết Jenkins và Architecture

Jenkins là một automation server độc lập (self-contained) được viết bằng Java. Nó hoạt động theo kiến trúc master-agent.

Architecture:



- **Master:** Kiểm soát toàn bộ hoạt động, scheduling, cấu hình.
- **Agent (Node):** Thực thi các job/build được master giao.
- **Plugin:** Mở rộng chức năng (SCM, Build tools, Notification...)
- **Job:** Tập hợp các task được định nghĩa để đạt mục tiêu cụ thể.

Jenkins lưu toàn bộ cấu hình, dữ liệu job, log... trong thư mục \$JENKINS_HOME.

7. Authorization (Phân quyền)

Authorization xác định các hành động mà một user được phép thực hiện trong Jenkins. Jenkins hỗ trợ nhiều chiến lược authorization khác nhau.

7.1 Matrix based authorization

- Permission được gán trên toàn instance (global).
- Kiểm soát chi tiết (fine-grained) ở cấp global.

1. Manage Jenkins -> Configure Global Security
2. Chọn "Matrix-based security"
3. Định nghĩa Global Permissions (Overall/Administer...)
4. Save

User	Overall	Read	Run Scripts	Configure	Administer
admin	✓	✓	✓	✓	✓
dev1	✓	✓	✓	✗	✗
tester	✓	✓	✓	✗	✗

7.2 Project based authorization

- Permission được định nghĩa ở cấp Job/Folder/Item.
- Permission khác nhau cho từng project.

1. Tạo hoặc mở một Job
2. Click "Configure" -> "Security"
3. Chọn "Project-based Matrix Authorization Strategy"
4. Thêm permission cho user/group
5. Save

Job: Build-App				
User	Build	Configure	Cancel	Read
dev1	✓	✓	✓	✓
tester	✓	✓	✗	✓
intern	✓	✓	✗	✗

6. Authentication

Authentication xác định cách user chứng minh danh tính. Jenkins hỗ trợ nhiều phương thức authentication.

Methods: Jenkins' own user database, LDAP, Active Directory, OAuth/Google Login/GitHub Login, và các OAuth provider khác.

8. Tổng kết

- ✓ CI giúp tích hợp code thường xuyên và bắt lỗi sớm.
- ✓ Jenkins tự động hóa build, test và deployment bằng kiến trúc master-agent.
- ✓ Tool có thể được quản lý tự động để đơn giản hóa build.

4. Jenkins Tool Management chi tiết

Jenkins có thể tự động cài đặt và quản lý các tool cần thiết cho build.

Managed Tools:

- JDK (Java Development Kit)
- Maven
- Gradle
- Git
- Ant
- Docker
- NodeJS
- Và nhiều tool khác...

Các bước cấu hình

1. Manage Jenkins
2. Global Tool Configuration
3. Select tool type
4. Add installation path hoặc để Jenkins tự cài (nếu hỗ trợ)
5. Save

5. Quản lý User trong Jenkins

Jenkins cho phép quản lý nhiều user khác nhau và cấp quyền truy cập phù hợp.

Các loại User:

- **Admin User:** Quản lý hệ thống Jenkins.
- **Regular User:** Tạo và chạy job.
- **Read-only User:** Xem job và build.

Các bước quản lý User:

1. Manage Jenkins -> Manage Users
2. Create User / Configure User
3. Đặt password và các thông tin khác
4. Gán roles/permissions
5. Save



TỔNG QUAN VỀ MAVEN

Maven là công cụ build automation và quản lý project, chủ yếu dành cho dự án Java. ☺

1. Cấu trúc Project Maven

Maven tuân theo một cấu trúc thư mục chuẩn giúp duy trì tính nhất quán giữa các project.

```
my-maven-project/
├── pom.xml → Project Object Model
├── src/
│   ├── main/
│   │   ├── java/ → Java source files
│   │   ├── resources/ → Resource files
│   │   └── test/
│   │       ├── java/ → Test source files
│   │       └── resources/ → Test resources
├── target/ → Compiled output, jars, reports
└── .mvn/ → Maven wrapper (tùy chọn)
```

Conventions → Configuration
(Viết ít XML hơn, code nhiều hơn!)

2. Maven Plugins

Plugin là trái tim của Maven. Mỗi plugin thực hiện một tác vụ được định nghĩa rõ ràng (compile, test, package, deploy,...).

Một số Plugin thông dụng:

Plugin	Mục đích
maven-compile-plugin	Biên dịch source code
maven-surefire-plugin	Chạy unit test
maven-jar-plugin	Tạo tệp JAR
maven-war-plugin	Tạo tệp WAR
maven-install-plugin	Cài artifact vào local repo
maven-deploy-plugin	Deploy artifact lên remote repo

Plugin được gắn (bound) vào các lifecycle phase để thực hiện goal.

3. Project Object Model (POM) - Đơn vị nền tảng của Project Maven

POM (pom.xml) là cốt lõi của project Maven. Nó chứa thông tin project, cấu hình build, dependency, plugin,...

Cấu trúc cơ bản của pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId> → Group / Organization
  <artifactId>my-maven-project</artifactId> → Tên Project
  <version>1.0.0</version> → Phiên bản Project
  <packaging>jar</packaging> → Kiểu đóng gói (jar/war)
  <name>My Maven Project</name> → Tên Project
  ... (dependencies, plugins, properties, ...)

</project>
```

4. Maven Build Lifecycle

Lifecycle là một chuỗi các phase. Mỗi phase có một mục tiêu cụ thể. Maven có 3 lifecycle cơ bản.

Lifecycle	Mục đích	Các Phase (theo thứ tự)
clean	Dọn dẹp project	pre-clean, clean, post-clean
default (build)	Build và deploy project	validate, initialize, generate-sources, process-sources, generate-resources, process-resources, compile, process-classes, generate-test-sources, process-test-sources, generate-test-resources, process-test-resources, test-compile, process-test-classes, test, prepare-package, package, verify, install, deploy
site	Tạo tài liệu project	pre-site, site, post-site, site-deploy

Luồng Default Lifecycle (Rút gọn)



Các Phase chính:

- validate** - Kiểm tra project hợp lệ và đầy đủ thông tin cần thiết.
- compile** - Biên dịch source code.
- test** - Chạy unit test bằng framework phù hợp.
- package** - Đóng gói code đã biên dịch theo định dạng phân phối (jar/war).
- install** - Cài package vào local repository.
- deploy** - Copy package cuối cùng lên remote repository.

5. Thêm External Dependencies vào pom.xml

Dependencies là các thư viện cần thiết cho project.

Thêm bên trong section
<dependencies>:

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope> <!-- tùy chọn -->
  </dependency>
</dependencies>
```

Scopes:

- compile** (mặc định) - Có sẵn khi compile và runtime
- provided** - Được cung cấp bởi JDK/container lúc runtime
- runtime** - Chỉ cần khi runtime
- test** - Chỉ dùng cho test class
- system** - Dùng jar do hệ thống cung cấp

Ghi chú: Transitive dependency được Maven tự động xử lý. Dùng "mvn dependency:tree" để xem cây dependency.

6. Build và Test Project Maven

Các lệnh Maven thông dụng (dùng CLI):

Command	Mục đích
mvn clean	Dọn dẹp thư mục target
mvn compile	Biên dịch source code
mvn test	Chạy unit test
mvn package	Đóng gói project (jar/war)
mvn install	Cài package vào local repository
mvn deploy	Deploy package lên remote repository
mvn clean install	Clean, build, test và install
mvn clean deploy	Clean, build, test và deploy

Ví dụ luồng Build:

mvn clean install → Code Compile → Tests Run → Package Created



JENKINS

Tạo Job và Cấu hình Build tự động

Jenkins là một automation server mã nguồn mở, dùng để tự động hóa việc build, test và deploy ứng dụng.

1. Jenkins Pipeline là gì?

Jenkins Pipeline là một bộ plugin hỗ trợ triển khai và tích hợp continuous delivery pipeline vào Jenkins.

Pipeline as Code: Định nghĩa delivery pipeline của bạn trong một "Jenkinsfile" và lưu trong source control.



2. Vì sao dùng Pipeline?

- Tự động hóa toàn bộ quy trình delivery.
- Build nhất quán và có thể lặp lại (repeatable).
- Dễ dàng tích hợp với SCM & các tool khác.
- Hỗ trợ các workflow phức tạp.
- Pipeline as Code - có thể version và review.

3. Tích hợp với GIT

- Pull source code từ Git repository.
- Phát hiện thay đổi và tự động kích hoạt build.
- Hỗ trợ Webhook để build ngay lập tức.



4. Bật Project-based Authorization cho một Job

- Vào Manage Jenkins → Configure Global Security
- Check "Enable project based matrix authorization"
- Save
- Vào job cụ thể → Configure → Permissions
- Đặt permission cho user/role chỉ áp dụng cho job này.

Vi dụ Permission

User/Role	Build	Configure	Read	Delete	Workspace
dev	✓	✓	✓	X	X
tester	✓	X	✓	X	X
deployer	✓	✓	✓	✓	✓

5. Source Code Management khi tạo Job

- New Item → Nhập tên job → Freestyle / Pipeline job
- Cuộn xuống "Source Code Management"
- Chọn "Git"
- Nhập Repository URL (vd: <https://github.com/user/repo.git>)
- Credentials (nếu là private repo)
- Branch Specifier (vd: */main)
- Save

SCM Options

Git, GitHub, Bitbucket, Subversion, CVS

6. Kích hoạt Automated Build

- Build periodically (cron expression)
- Poll SCM
- GitHub Hook Trigger cho GITScm polling
- Build sau khi project khác được build
- Build khi có thay đổi push lên Git (dùng Webhook)

Vi dụ (Cron)

H/15 * * * * *

→ Build mỗi 15 phút

7. Cấu hình Maven Job

- Tạo một Freestyle project
- SCM → Git (thêm thông tin repo)
- Build Triggers → Chọn trigger cần thiết
- Build → Add build step → Invoke top-level Maven targets
 - Goals: clean install (hoặc tùy chỉnh)

Vi dụ Maven Goal

clean, compile, test, package, install

8. Post Build Options

Post Build action được thực thi sau khi build hoàn tất.

Option	Mô tả
Email Notification	Gửi email khi build success/failure
Trigger another build	Kích hoạt downstream job sau khi hoàn tất
Publish JUnit test result	Publish báo cáo test (JUnit XML)
Publish HTML reports	Publish báo cáo HTML
Archive the artifacts	Lưu artifact được tạo bởi build
Slack / MS Teams Notification	Gửi trạng thái build đến Slack / Teams
Cobertura / JaCoCo Report	Publish báo cáo code coverage
Set build status	Cập nhật trạng thái build (Stable/Unstable)

9. Thêm Slave Node vào Jenkins

- Manage Jenkins → Manage Nodes and Clouds
- Click "New Node"
- Nhập Node Name
- Chọn "Permanent Agent"
- Đặt Number of Executors
- Đặt Remote Root Directory
- Launch method (Launch agent via JNLP/SSH)
- Save
- Start the Node

Architecture



10. Xây dựng Delivery Pipeline

Luồng ví dụ:



Sample Jenkins Pipeline (Jenkinsfile)

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { git 'https://github.com/user/repo.git' } }
    stage('Build') { steps { sh 'mvn clean install' } }
    stage('Test') { steps { sh 'mvn test' } }
    stage('Package') { steps { sh 'mvn package' } }
    stage('Deploy') { steps { sh './deploy.sh' } }
  }
  post {
    always { mail to: 'devteam@company.com', subject: 'Build Status', body: 'Pipeline completed' } }
  }
}
```

11. Cài đặt Notification trong Jenkins

- Email Notification
 - Manage Jenkins → Configure System → Extended E-mail Notification
 - Thêm SMTP server, test cấu hình, save
 - Dùng trong Post-build actions của job
- Slack / MS Teams Notification (dùng Plugin)
- Build status trên dashboard

Email Triggers

Always, Failure, Unstable, Success, Not built

12. Quản lý Plugin trong Jenkins

- Manage Jenkins → Manage Plugins
- Available: danh sách plugin có thể cài
- Installed: danh sách plugin đã cài
- Updates: các bản cập nhật plugin
- Advanced: Upload .hpi file, Update Site

Plugin hữu ích

Pipeline, Git, Maven Integration, Publish Over SSH, Email Extension, Slack Notification, JUnit, SonarQube Scanner



DOCKER

Docker là một nền tảng sử dụng công nghệ containerization để build, ship và run ứng dụng nhất quán ở mọi môi trường. ☺

1. GIỚI THIỆU

1.1 Thách thức thực tế của vận chuyển hàng hóa (Shipping)

Hãy tưởng tượng một công ty vận chuyển giao hàng từ nơi này đến nơi khác.

- Nhiều loại xe tải, tàu, máy bay khác nhau về kích thước.
- Hàng hóa được đóng gói thủ công → Tốn thời gian & công sức.
- Rủi ro hư hỏng hoặc thất lạc khi vận chuyển.
- Quy định khác nhau ở mỗi cảng → Chậm trễ.
- Khó theo dõi và quản lý mọi lô hàng.
- Chi phí cao và quy trình kém hiệu quả.

1.2 Giới thiệu Docker và công nghệ của nó

- Docker giải quyết các thách thức này bằng containerization.
- Nó đóng gói một ứng dụng và toàn bộ dependency của nó vào một đơn vị duy nhất gọi là container.
- Cùng một container có thể chạy ở bất kỳ đâu - trên laptop, server hoặc cloud.



Lợi ích:

- ☆ Nhẹ và nhanh
- ☆ Nhất quán trên mọi môi trường
- ☆ Dễ deploy và scale
- ☆ Sử dụng tài nguyên tốt hơn

1.3 Tìm hiểu về Docker Image và Container

Docker Image: Một template chỉ đọc (read-only) chứa application code, runtime, thư viện, biến môi trường và tệp cấu hình.

Image	Container
Template (chỉ đọc)	Instance đang chạy (đọc-ghi)
Tạo bằng Dockerfile	Tạo từ Image
Có thể chia sẻ	Tồn tại tạm thời (có thể stop/remove)
Lưu trong Docker Hub	Chạy trên Docker Engine

Điểm mấu chốt:

Image giống như bản thiết kế (blueprint) của một căn nhà.
Container là căn nhà thực tế được xây từ bản thiết kế đó.

2. LÀM VIỆC VỚI CONTAINER

2.1 Cách chia sẻ và sao chép một Container

- Bạn có thể tạo một image mới từ container có sẵn và chia sẻ nó với người khác.



```
# Commit một container thành image
docker commit <container_id> <image_name>:<tag>

# Gắn tag cho image (tùy chọn)
docker tag <image_name>:<tag> <dockerhub_username>/<image_name>:<tag>

# Push lên Docker Hub
docker push <dockerhub_username>/<image_name>:<tag>
```

2.2 Vòng đời Container (Container Life Cycle)



Các Command thông dụng:

docker create <image>	→ Tạo một container
docker start <container>	→ Khởi động container
docker pause <container>	→ Tạm dừng container đang chạy
docker unpause <container>	→ Bỏ tạm dừng container
docker stop <container>	→ Dừng container
docker restart <container>	→ Khởi động lại container
docker rm <container>	→ Xóa container đã dừng

2.3 Cách dùng Base Image và tùy chỉnh

Base image là điểm khởi đầu. Ta tùy chỉnh nó bằng cách thêm application, thư viện, config,...

Vi dụ: Dùng ubuntu làm base image và cài Nginx

```
FROM ubuntu:22.04 # Base Image
RUN apt-get update && apt-get install -y nginx # Tùy chỉnh
COPY index.html /var/www/html/index.html # Copy tệp
EXPOSE 80 # Mở port
CMD ["nginx", "-g", "daemon off;"] # Chạy lệnh
```

2.4 Tạo Docker File

Dockerfile chứa các chỉ thị (instruction) để build một Docker image.

Vi dụ Dockerfile cơ bản:

```
FROM python:3.11-slim # Base Image
WORKDIR /app # Đặt thư mục làm việc
COPY requirements.txt . # Copy tệp dependency
RUN pip install -r requirements.txt # Cài dependency
COPY . . # Copy source code
EXPOSE 5000 # Mở port
CMD ["python", "app.py"] # Chạy ứng dụng
```

2.5 Cách Publish Image lên Docker Hub

- Tạo tài khoản tại hub.docker.com
- Đăng nhập bằng CLI → `docker login`
- Gắn tag cho image → `docker tag <image> <username>/<image>:<tag>`
- Push image → `docker push <username>/<image>:<tag>`

Tổng kết

- ✓ Docker giải quyết các thách thức vận chuyển thực tế bằng cách containerize ứng dụng.
- ✓ Image là template, Container là instance đang chạy.
- ✓ Container trải qua một vòng đời: Create → Start → Running → Stop → Remove.
- ✓ Ta có thể tùy chỉnh base image, tạo Dockerfile và publish lên Docker Hub.



GIỚI THIỆU VỀ DOCKER NETWORKING

Docker sử dụng công nghệ containerization để build, ship và run ứng dụng nhất quán ở mọi môi trường. ☺

1. Các loại Network trong Docker

Docker cung cấp nhiều network driver. Dưới đây là các loại network mặc định:

Loại	Driver	Mô tả	Trường hợp đúng
bridge	bridge	Network mặc định. Container cùng bridge network có thể giao tiếp.	Giao tiếp trên một host
host	host	Container dùng chung network namespace với host.	Ứng dụng hiệu năng cao
none	null	Container không có network.	Container biệt lập
overlay	overlay	Kết nối nhiều Docker daemon lại với nhau.	Giao tiếp đa host (Swarm)
macvlan	macvlan	Gán địa chỉ MAC cho container, khiến nó xuất hiện như thiết bị vật lý.	Ứng dụng củ cần network vật lý
ipvlan	ipvlan	Tương tự macvlan nhưng hiệu quả hơn.	Triển khai quy mô lớn

Ứng dụng: Tạo network riêng biệt hoặc dùng chung giữa các container, hoặc giữa container và thế giới bên ngoài.

2. Docker Container Networking

- Mặc định, mọi container được kết nối vào bridge network.
- Container trong cùng network có thể giao tiếp bằng tên container.
- Container ở network khác nhau không thể giao tiếp trừ khi được cho phép rõ ràng.
- Bạn có thể tạo custom network để cô lập và kết nối container.

Ví dụ: Giao tiếp trong Bridge Network



Các Command hữu ích:

docker network ls	-- Liệt kê mọi network
docker network create my_net	-- Tạo custom network
docker network inspect my_net	-- Xem chi tiết network
docker run --network my_net nginx	-- Chạy container trong network
docker network connect my_net container	-- Kết nối container đang chạy
docker network disconnect my_net container	-- Ngắt kết nối container

3. Docker Compose - Giới thiệu

- Docker Compose là công cụ để định nghĩa và chạy ứng dụng Docker multi-container.
- Nó dùng tệp YAML (docker-compose.yml) để cấu hình các service của ứng dụng.
- Chỉ với một lệnh, bạn có thể tạo và khởi động mọi service.

Mẫu docker-compose.yml (cho php, wordpress, mysql)

```
version: '3.8'
services:
  db:
    image: mysql:5.7           # MySQL Service
    restart: always            # Tự khởi động lại
    environment:               # Biến môi trường
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:                   # Lưu trữ dữ liệu
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest    # WordPress Service
    restart: always
    depends_on:                # Phụ thuộc vào db
      - db
    ports:                     # Port mapping
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress

  phpmyadmin:
    image: phpmyadmin/phpmyadmin # PHPMyAdmin Service
    restart: always
    ports:
      - "8081:80"              # Port mapping
    environment:
      PMA_HOST: db
      MYSQL_ROOT_PASSWORD: rootpass
    volumes:
      - db_data:               # Named volume
```

Command hữu ích:

- docker-compose up -d → Khởi động mọi service ở chế độ nền
- docker-compose down → Dừng và xóa container, network, volume
- docker-compose ps → Liệt kê service đang chạy
- docker-compose logs -f → Xem log
- docker-compose build → Rebuild service

Lợi ích của Compose:

- ✓ Cấu hình dễ dàng
- ✓ Môi trường tái tạo được
- ✓ Quản lý ứng dụng multi-container
- ✓ Hoạt động ở development, test & production

4. Docker Swarm - Giới thiệu

- Docker Swarm là công cụ clustering và orchestration gốc của Docker.
- Nó biến một tập hợp Docker host thành một Docker engine ảo duy nhất.
- Nó cung cấp: High availability, Scaling, Load balancing, Self-healing.

Swarm Architecture



5. Dùng Docker Compose để tạo php, wordpress, mysql

Ta có thể dùng tệp docker-compose.yml ở mục 3).

- docker-compose up -d → Tạo và khởi động php, wordpress, mysql
- Truy cập WordPress: <http://<host-ip>:8080>
- Truy cập phpMyAdmin: <http://<host-ip>:8081>

6. Khởi động Container trên Cluster với Docker Swarm

- Khởi tạo Swarm trên Manager Node: `docker swarm init`
- Cho Worker Node tham gia Swarm: `docker swarm join --token <token> <manager-ip>:2377`
- Xác minh Node: `docker node ls`
- Deploy Service (từ Compose file): `docker stack deploy -c docker-compose.yml myapp`

7. Tạo và Scale Application trong Docker Swarm

Tạo Application (Stack)

`docker stack deploy -c docker-compose.yml myapp`
Lệnh này tạo mọi service định nghĩa trong compose file dưới dạng swarm service.

Scaling Services

`docker service scale myapp_wordpress=3`



Command khác

`docker service scale myapp_wordpress=5`
`docker service update --image wordpress:latest myapp_wordpress`
`docker stack rm myapp`



KUBERNETES

Tự động hóa ứng dụng đã container hóa ở quy mô lớn

Kubernetes (K8s) là một nền tảng container orchestration mà nguồn mở, tự động hóa việc deploy, scale và quản lý ứng dụng đã container hóa. ☺

1. Giới thiệu về Kubernetes

Kubernetes (K8s) là một hệ thống mã nguồn mở để tự động hóa việc deploy, scale và quản lý ứng dụng đã container hóa.

Tính năng chính:

- Deploy và rollback tự động
- Horizontal scaling và self-healing
- Service discovery và load balancing
- Sử dụng tài nguyên hiệu quả
- Cấu hình khai báo (desired state)
- Portable trên mọi môi trường (cloud/on-prem)

K8s vs Docker

Docker → Container Runtime (tạo và chạy container)

Kubernetes → Container Orchestrator (quản lý container ở quy mô lớn)

3. Khái niệm Pods, ReplicaSets, Deployments, Namespaces

► **Pod:** Đơn vị deploy nhỏ nhất trong K8s. Một pod có thể chứa một hoặc nhiều container dùng chung network và storage.



► **ReplicaSet:** Đảm bảo luôn có đủ số lượng pod replica đang chạy.



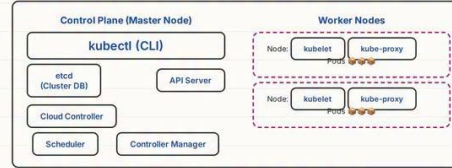
► **Deployment:** Quản lý ReplicaSet và cung cấp cập nhật khai báo (rolling update, rollback, scaling).



► **Namespace:** Cung cấp cách chia tài nguyên cluster giữa nhiều user hoặc team.



2. Tổng quan kiến trúc Kubernetes Cluster



- **API Server:** Frontend cho control plane của K8s.
- **etcd:** Key-value store nhất quán, tính sẵn sàng cao.
- **Scheduler:** Lên lịch pod cho worker node.
- **Controller Manager:** Chạy các controller process (ReplicaSet, Node, Endpoints,...)
- **kubelet:** Đảm bảo container đang chạy trong một pod.
- **kube-proxy:** Network proxy, duy trì network rule trên node.

4. Khái niệm Services và Networking

Service: Cung cấp một endpoint ổn định (IP/hostname) để truy cập một nhóm pod.

Các loại Service:

- **ClusterIP:** IP nội bộ. Chỉ truy cập được trong cluster. (mặc định)
- **NodePort:** Expose service trên IP của mỗi Node ở một port cố định.
- **LoadBalancer:** Expose service ra ngoài bằng LB của cloud provider.
- **ExternalName:** Map service tới một tên DNS bên ngoài.



Khái niệm Networking:

- Mỗi pod có địa chỉ IP riêng.
- Pod có thể giao tiếp với nhau mà không cần NAT.
- Service cung cấp networking ổn định và load balancing.
- CNI (Container Network Interface) plugin quản lý networking, vd: Calico, Flannel, Weave, Cilium.

5. Persistent Volumes và Persistent Volume Claims

Kubernetes cung cấp persistent storage bằng PV và PVC.

► **Persistent Volume (PV):** Một phần storage trong cluster, được cấp bởi admin hoặc tự động.



► **Persistent Volume Claim (PVC):** Một yêu cầu storage từ user.



PVC đóng vai trò là một yêu cầu và bind với PV. Pod sử dụng storage đã claim để lưu dữ liệu bên vững ngay cả sau khi restart.

6. Thiết kế Pod

- Pod nên được thiết kế chạy một tiến trình chính trên mỗi container (best practice).
- Chỉ dùng pod multi-container khi các container gắn kết chặt chẽ.

Các pattern:



7. Labels, Selectors, Jobs, Schedulers

Labels và Selectors

Labels: Cặp Key-Value gắn vào object.
vd: app=web, env=prod

Selectors: Dùng để lọc resource theo label. vd:
matchLabels: {app: web}

Jobs

Job tạo ra một hoặc nhiều pod và đảm bảo một số lượng pod cụ thể hoàn tất thành công.

Scheduler

Scheduler gán pod cho node dựa trên resource, ràng buộc và chính sách (CPU, Memory, GPU, taint/toleration...).

Điểm chính cần nhớ

- ☞ Kubernetes tự động hóa các thao tác container.
- ☞ Dựa trên cấu hình khai báo và desired state.
- ☞ Khả năng mở rộng, portable và có khả năng phục hồi.



ANSIBLE

Tự động hóa thật đơn giản!

Ansible là công cụ tự động hóa mã nguồn mở dùng cho configuration management, application deployment, task automation và orchestration.

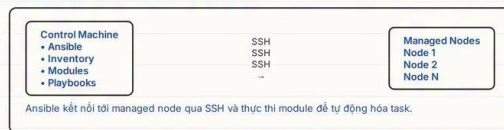
1. Giới thiệu Ansible - Công cụ Configuration Management

Ansible là agentless, dễ học và dùng cú pháp YAML đơn giản để tự động hóa.

Cơ bản / Những gì sẽ được cài đặt

- Ansible (trên control machine)
- Python (bắt buộc trên cả control và managed node)
- SSH (dùng để giao tiếp)
- Không cần cài agent trên managed node.

Tìm hiểu Ansible Architecture



Yêu cầu Control Machine

- OS: Linux / macOS / Windows (khuyến nghị WSL)
- Python: 3.8 trở lên
- Ansible: Phiên bản stable mới nhất
- SSH access đến managed node
- Đủ dung lượng đĩa và memory

Yêu cầu Managed Node

- OS: Bất kỳ (Linux, Windows)
- Python: 2.7+ hoặc 3.5+ (đã cài sẵn)
- SSH server phải đang chạy (Linux)
- WinRM được bật (Windows)
- User có quyền sudo/Administrator

2. Inventory

Inventory định nghĩa danh sách host và group mà Ansible quản lý.

Mẫu tệp Inventory (hosts.ini)

```
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[dbservers]
db1 ansible_host=192.168.1.20
db2 ansible_host=192.168.1.21

[allservers:children]
webservers
dbservers
```

Hosts và Groups

- Host là các máy riêng lẻ.
- Group là một tập hợp các host.
- Children group cho phép lồng (nesting) các group.



Host Variables

Biến (variable) áp dụng riêng cho một host cụ thể.

```
# host_vars/web1.yml
ansible_user: ubuntu
ansible_port: 22
app_env: production
... Tệp này chứa biến chỉ dành cho host web1.
```

Group Variables

Biến áp dụng cho mọi host trong một group.

```
# group_vars/webservers.yml
http_port: 80
max_clients: 200
app_env: staging
... Các biến này sẽ có sẵn cho mọi host trong group webservers.
```

3. Tìm hiểu các Ansible Module

Module là các khối xây dựng (building block) của Ansible. Chúng dùng để thực hiện task cụ thể trên managed node.

Nhóm	Module	Mục đích	Ví dụ sử dụng
System	ping	Kiểm tra kết nối	ansible all -m ping
	command	Chạy command trên remote node	ansible all -m command -a "uptime"
	shell	Chạy shell command	ansible all -m shell -a "df -h"
	setup	Thu thập facts về host	ansible all -m setup
File	copy	Copy tệp đến remote node	ansible web -m copy -a "src=index.html dest=/var/www/html/"
	file	Quản lý tệp và thư mục	ansible all -m file -a "path=/tmp/test state=absent"
Package	template	Deploy template (Jinja2)	ansible all -m template -a "src=app.j2 dest=/opt/app.conf"
	apt	Quản lý gói apt (Debian/Ubuntu)	ansible all -m apt -a "name=nginx state=present"
Service	yum	Quản lý gói yum (RHEL/CentOS)	ansible all -m yum -a "name=httpd state=latest"
	service	Quản lý service	ansible all -m service -a "name=nginx state=started"
User	systemd	Quản lý systemd service	ansible all -m systemd -a "name=nginx enabled=yes"
	user	Quản lý tài khoản user	ansible all -m user -a "name=devops state=present"
Khác	group	Quản lý group	ansible all -m group -a "name=devops state=present"
	git	Clone/quản lý git repository	ansible all -m git -a "repo=https://github.com/..."
	cron	Quản lý cron job	ansible all -m cron -a "name=backup minute=0 hour=2 job=/script.sh"
	firewalld	Quản lý firewall rule	ansible all -m firewalld -a "service=http state=enabled"
	script	Chạy local script trên remote node	ansible all -m script -a "backup.sh"

Ghi chú:

Ansible là agentless và dùng SSH/WinRM để giao tiếp.

Mẫu Ad-hoc Command

ansible all -m ping

Tự động hóa mọi thứ ☺

GIÁM SÁT VÀ QUAN SÁT HỆ THỐNG (MONITORING AND OBSERVABILITY)

Logging và Metrics: Prometheus, Grafana và ELK stack để theo dõi ứng dụng và cảnh báo hệ thống

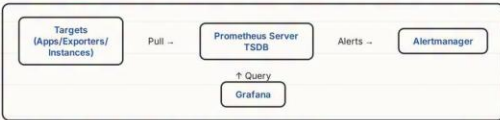
1. Tổng quan Logging và Metrics

- Monitoring cho ta biết điều gì đang xảy ra (WHAT).
- Observability giúp ta hiểu vì sao nó xảy ra (WHY).
- Metrics = Dữ liệu số theo thời gian (CPU, memory, requests)
- Logs = Các sự kiện rời rạc (lỗi, truy cập, trace)
- Traces = Luồng request qua các service (Jaeger, Zipkin)

2. PROMETHEUS (Thu thập Metrics)

- Bộ công cụ monitoring và alerting mã nguồn mở.
- Mô hình Pull-based: Prometheus scrape metric từ target.
- Lưu dữ liệu trong time-series database.
- Dùng PromQL để truy vấn.

Kiến trúc Prometheus



Thành phần chính

- Prometheus Server:** thu thập & lưu trữ metric
- Exporters:** expose metric (node_exporter, app_exporter)
- Alertmanager:** xử lý alert và notification
- Grafana:** visualization và dashboard

3. GRAFANA (Visualization)

- Nền tảng analytics & visualization mã nguồn mở.
- Kết nối với nhiều data source (Prometheus, InfluxDB, MySQL)
- Xây dựng dashboard với panel, alert, annotation.
- Chia sẻ dashboard và cấp quyền cho user.

Tính năng:

- ✓ Dashboard phong phú
- ✓ Alert & notification (Email, Slack, Telegram, Webhook)
- ✓ Hỗ trợ variable và templating
- ✓ Hệ sinh thái plugin đa dạng

4. ELK STACK (Quản lý Log)

ELK = Elasticsearch + Logstash + Kibana

- Centralized logging và phân tích log.
- Hỗ trợ theo dõi application log và tạo alert.



Thành phần:

- Logstash:** Thu thập log từ nhiều nguồn, parse & filter.
- Elasticsearch:** Lưu trữ, index và giúp log có thể tìm kiếm.
- Kibana:** Trực quan hóa log qua dashboard và biểu đồ.

Trường hợp sử dụng phổ biến:

- ✓ Theo dõi lỗi ứng dụng
- ✓ Giám sát hoạt động người dùng
- ✓ Phân tích sự kiện bảo mật
- ✓ Audit và tuân thủ hệ thống



5. CẢNH BÁO HỆ THỐNG (SYSTEM ALERTS)

- Alert thông báo cho team về sự cố trước khi user bị ảnh hưởng.
- Alertmanager (trong Prometheus) xử lý việc định tuyến alert.
- Có thể tích hợp với Email, Slack, PagerDuty, Webhook,...

Ví dụ Alert Rule (Prometheus Alert)

```
groups:
- name: high_cpu_alerts
  rules:
  - alert: HighCPUUsage
    expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
    for: 5m
    labels:
    severity: critical
    annotations:
    summary: "High CPU usage detected on {{ $labels.instance }}"
    description: "CPU usage is above 80% for more than 5 minutes."
```



DEVSECOPS

Quét lỗ hổng bảo mật, quản lý secret và triển khai bảo mật shift-left

DEV  OPS

1. QUÉT LỖ HỔNG BẢO MẬT (VULNERABILITY SCANNING)

- Xác định lỗ hổng bảo mật sớm trong SDLC.
- Quét code, dependency, container, hạ tầng & image.
- Giúp khắc phục lỗ hổng trước khi lên production.

Loại	Tool	Đối tượng quét	Ví dụ
SAST	SonarQube, Checkmarx	Source Code	SQL Injection, XSS
SCA	OWASP Dependency-Check, NPM Audit, Snyk	Dependencies	Thư viện lỗi thời/lỗ hổng
DAST	OWASP ZAP, Burp Suite	Ứng dụng đang chạy	SQLi, CSRF
Container Scan	Trivy, Clair, Anchore	Docker Images	CVE, Misconfiguration
IaC Scan	Checkov, tfsec	Terraform, CloudFormation, K8s Manifest	Cấu hình sai

Best Practices

- ✓ Chạy scan trong CI pipeline
- ✓ Fail build khi có vấn đề nghiêm trọng
- ✓ Tạo report & theo dõi trong dashboard
- ✓ Scan lại sau khi fix



3. SHIFT-LEFT SECURITY

- Tích hợp bảo mật ngay từ sớm trong vòng đời phát triển.
- Fix lỗi sớm → Tiết kiệm chi phí, nhanh hơn, an toàn hơn.
- Bảo mật là trách nhiệm của tất cả mọi người.



Các hoạt động bảo mật (Shift-Left)



2. QUẢN LÝ SECRET (SECRETS MANAGEMENT)

- Lưu và quản lý dữ liệu nhạy cảm an toàn (mật khẩu, key, token, cert).
- Không bao giờ hardcode secret trong code hoặc config.
- Sử dụng công cụ quản lý secret chuyên dụng.

Tool	Tính năng
HashiCorp Vault	Secret tập trung, dynamic secret, audit log
AWS Secrets Manager	Dịch vụ quản lý lưu trữ và xoay vòng secret
Azure Key Vault	Lưu key, secret, cert trên Azure
Google Secret Manager	Lưu trữ an toàn tích hợp IAM

Ví dụ (Vault)

```
# Lưu một secret
vault kv put secret/db password="MyPass@123"
```

```
# Đọc một secret
vault kv get secret/db
```

Best Practices

- ✓ Lưu secret ngoài code
- ✓ Xoay vòng (rotate) secret định kỳ
- ✓ Dùng Least Privilege Access
- ✓ Bật audit logging

4. LỢI ÍCH CỦA DEVSECOPS

- ✓ Phát hiện sớm lỗ hổng bảo mật
- ✓ Giảm rủi ro bảo mật
- ✓ Tuân thủ và quản trị tốt hơn
- ✓ Feedback nhanh hơn & release an toàn hơn
- ✓ Tự động hóa và nhất quán

Điểm chính cần nhớ

- Giám sát mọi thứ (metric, log, trace)
- Dùng đúng tool: Prometheus, Grafana, ELK
- Cảnh báo trước khi có ảnh hưởng
- Bảo mật sớm, bảo mật luôn luôn
- Tự động hóa bảo mật trong mọi pipeline